

嵌入式系统实验指导书

基于 CT117E-M4 开发板

2026 年修改版

自动化系

新疆大学

目 录

实验1 最小系统PCB设计	2
实验2 LED流水灯	7
实验3 UART异步串行通信.....	16
实验4 GPIO应用开发1	18
实验5 GPIO应用开发2	20
实验6 可编程延迟模块（基本定时功能）	22
实验7 可编程延迟模块（频率输出及输入捕获）	23
实验8 数模转换模块	24

实验1 最小系统PCB设计

1.1 实验目的

了解嵌入式系统的最小系统设计方法
掌握 Altium Designer 软件绘制最小系统 PCB

1.2 实验设备及器件

计算机，Altium Designer 软件

1.3 实验流程

(1) 原理图绘制流程：



① 打开软件。在桌面或开始菜单中找到Altium Designer 软件图标并打开软件。

②新建工程。依次点击菜单栏：文件→新的→项目，在弹出的对话框中选择保存工程的位置，点击“Create”按钮。

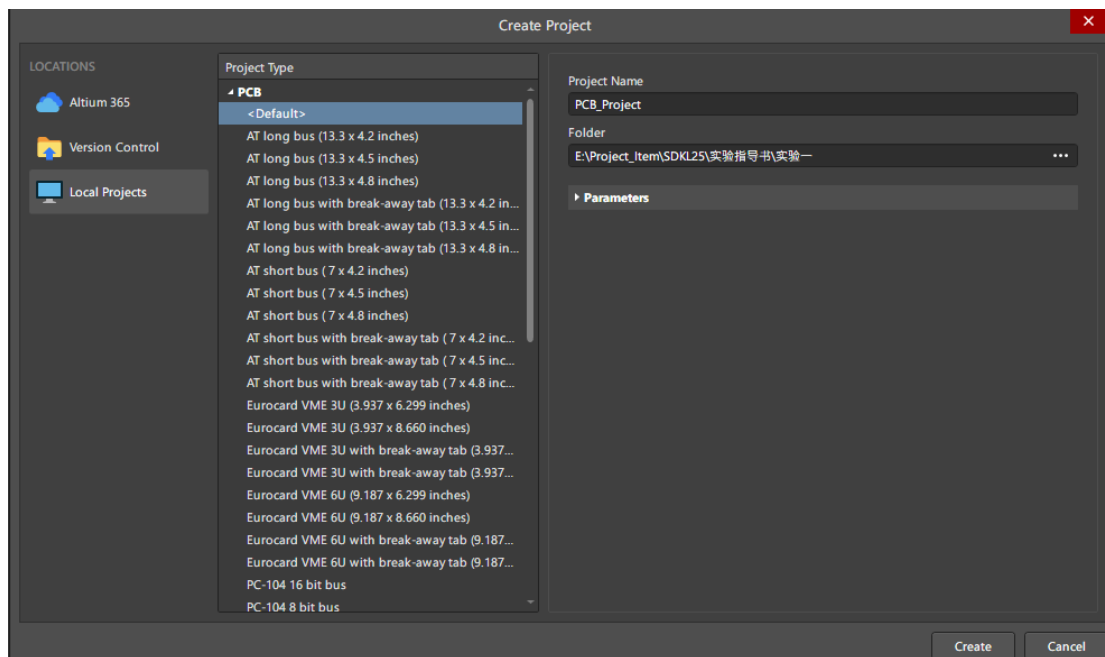


图1.1 Create Project选项卡

③新建原理图。依次点击菜单栏：文件→新的→原理图，即看到原理图已经添加进工程，点击 Ctrl+S 按键可以保存原理图。

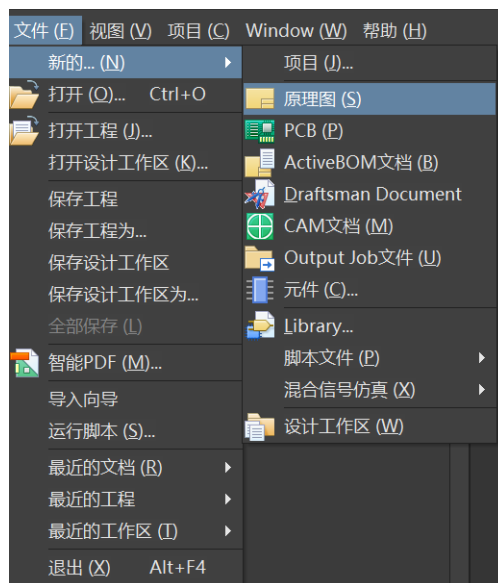


图1.2 新建原理图

④添加实验1参考资料中提供的原理图库和 PCB 库，在左侧的工程栏上点击鼠标右键选择：添加已有文件到工程

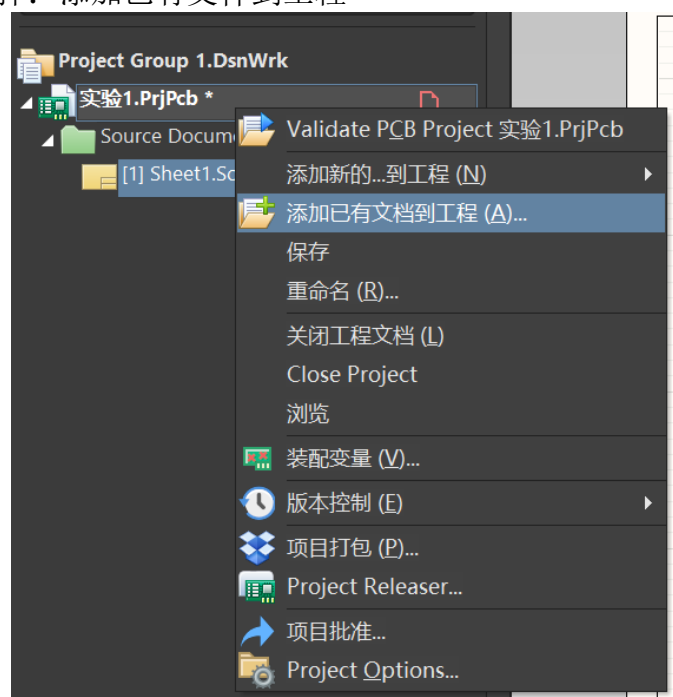


图1.3 添加库文件菜单

在弹出的对话框中选择本次实验所提供的原理图库和 PCB 库，将其添加到工程中，添加后如图所示，可以在工程中看到所添加的原理图库和PCB库文件：

经添加进工程，点击 **Ctrl+S** 按键可以保存 PCB 图。

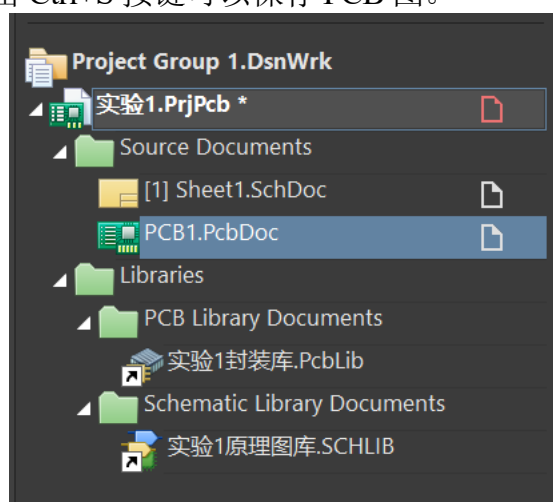


图1.7 添加PCB文件并保存后的工程目录

②将原理图导入到 PCB 中。依次点击菜单栏：设计→Update PCB…。在弹出的对话框中依次点击“验证变更”、“执行变更”，若原理图无误，则可以在 PCB 文件中看到对应的元器件已导入。

③参考书本中的系统电路，排放元器件，进行自动布线完成 PCB 设计。

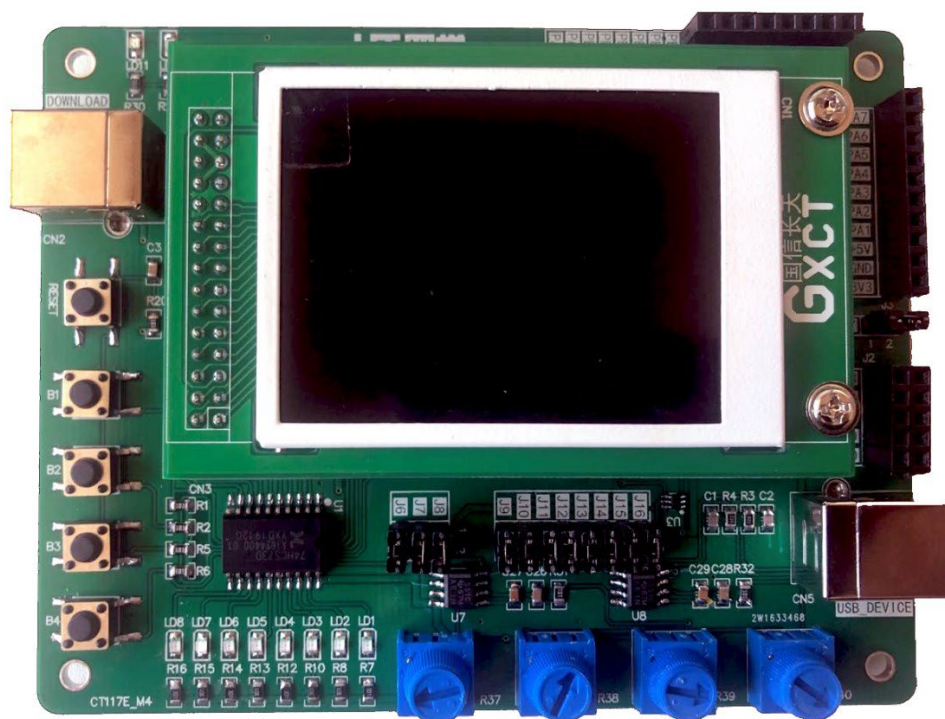


图1.7 CT117T-M4开发板实物图

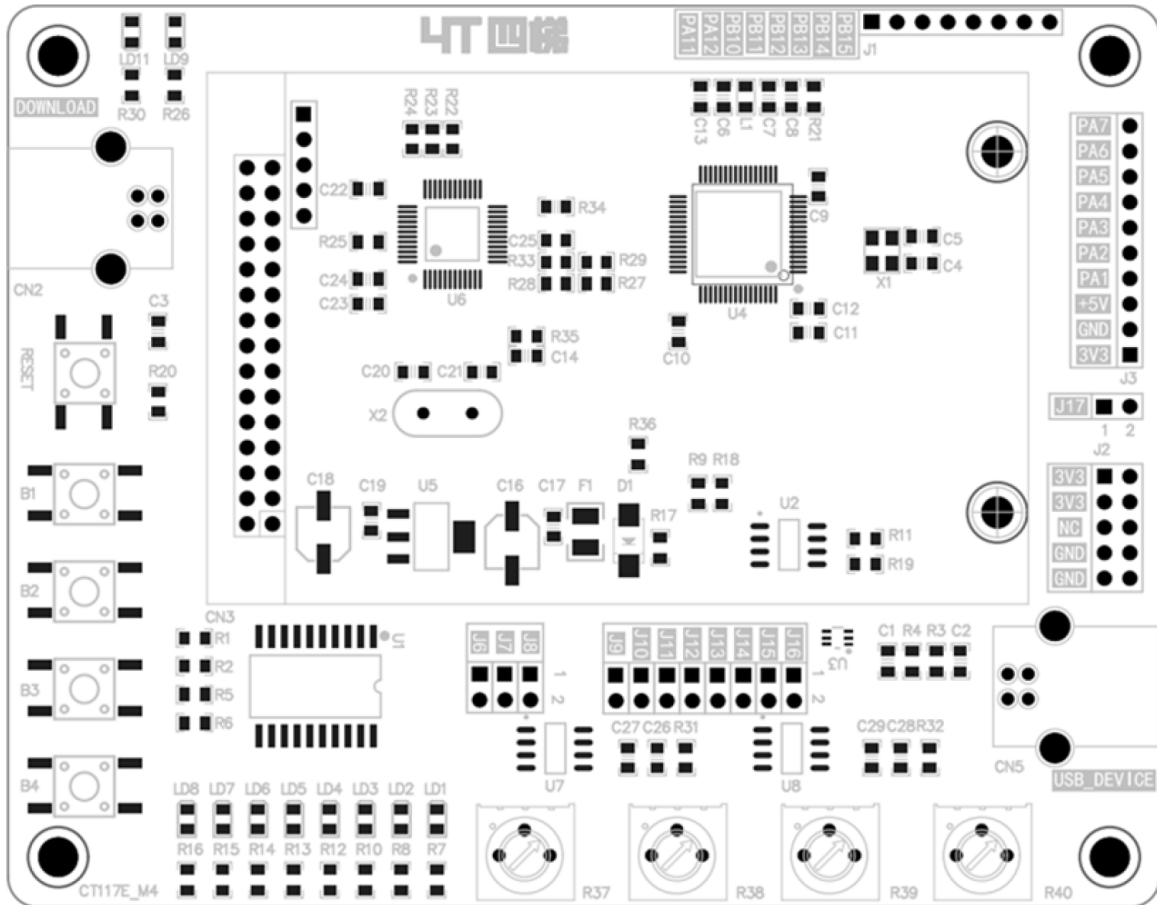


图1.8 CT117T-M4开发板硬件布局图

1.4 实验思考题

- 1、什么是单片机的最小系统，最小系统的作用是什么。
- 2、请简述 PCB 设计的一般流程。

实验2 LED流水灯

2.1 实验目的

掌握STM32集成开发环境软件的基本使用方法和软件调试方法
了解STM32 Cortex-M4系列芯片的软件开发流程

2.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

2.3 实验流程

2.3.1 通过STM32 CubeMX软件创建一个STM32工程:

(1) 打开STM32CubeMX软件，熟悉软件界面。

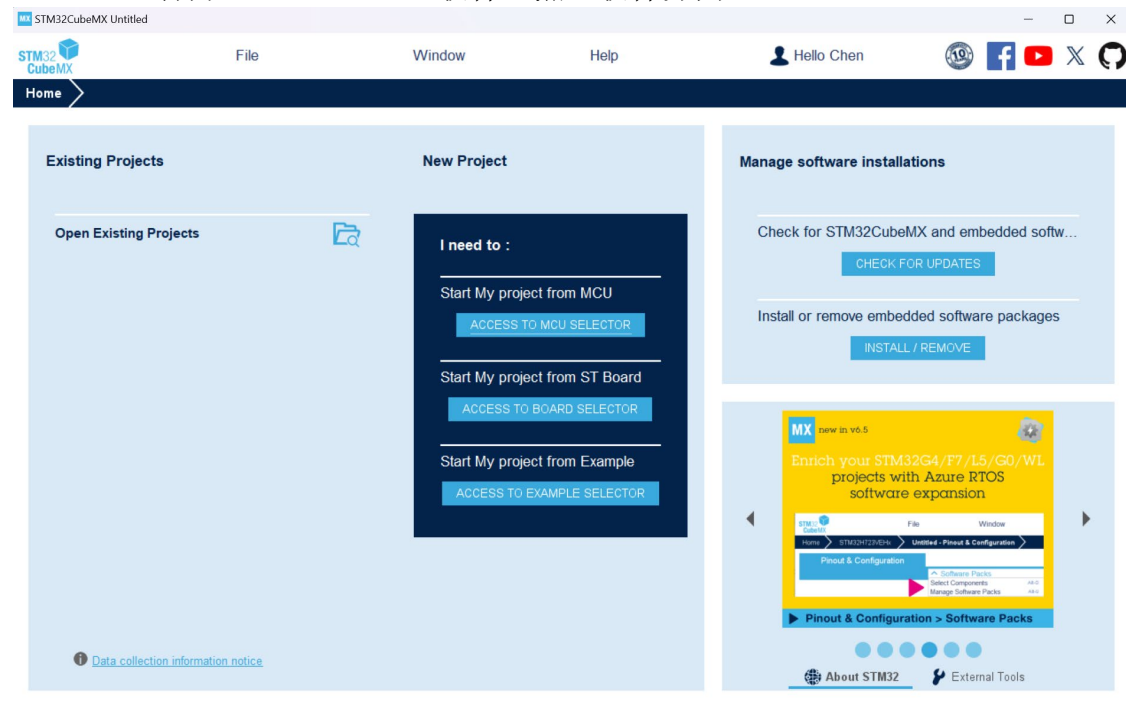


图2.1 STM32CubeMX 软件界面

(2) 导入STM32G431资源包（实验资源/STM32CubeMX/ en.stm32cubeg4_v1-5-0.zip），实验所用资源包版本为1.5.0版；待软件打开后，点击右侧Install or remove embedded software packages，在弹出的界面中选择From Local...按钮，选中电脑上的离线资料包en.stm32cubeg4_v1-5-0.zip进行安装。（因机房未联网，在这里采用离线安装的方法，在已联网的电脑可以直接在包管理界面找到STM32G431资料包，选择1.5.0版本直接在线安装即可。）

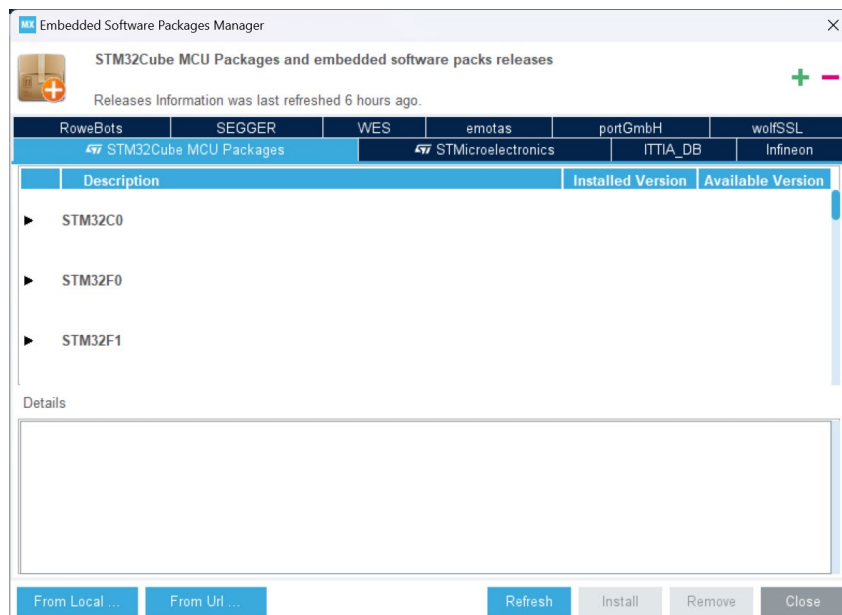


图2.2 安装软件包界面

(3) 新建一个STM32G431芯片的CubeMX工程。点击STM32CubeMX软件主界面上的“ACCESS TO MCU SELECTOR”按钮，在弹出的芯片选择界面中，找到Commercial Part Number输入框，在输入框中输入STM32G431RBT6，在右侧的选择结果中选中STM32G431RBT6，点击右上角的“Start Project”创建工程。

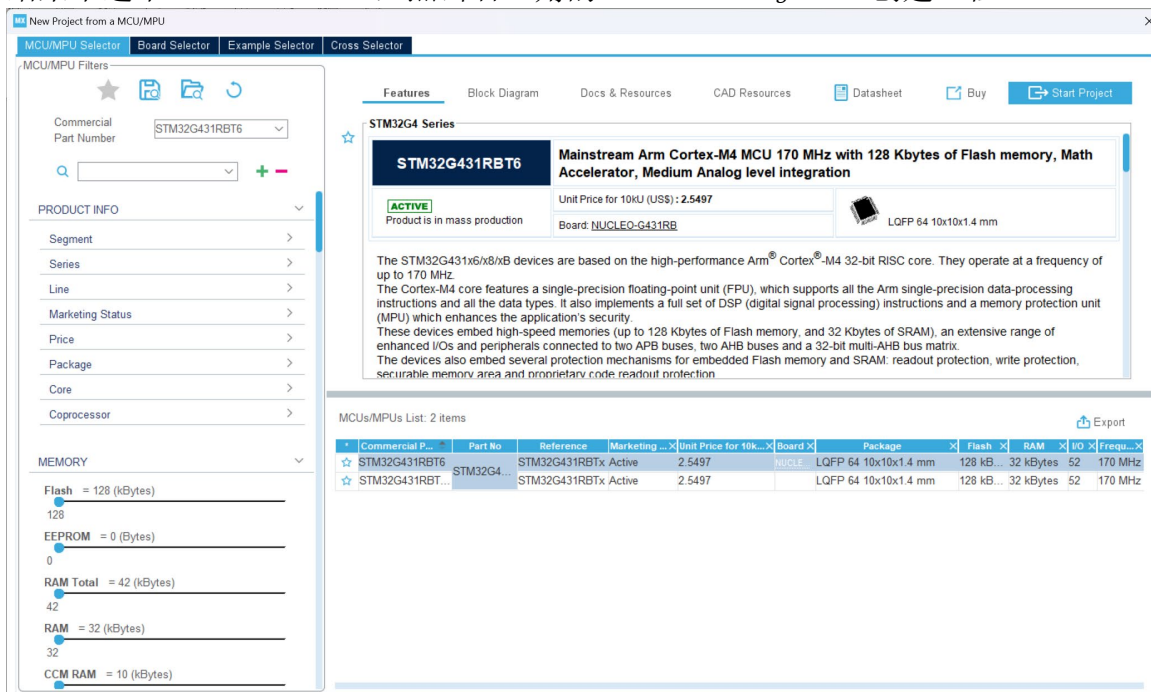


图2.3 芯片选择界面

(4) 熟悉CubeMx工程的界面。

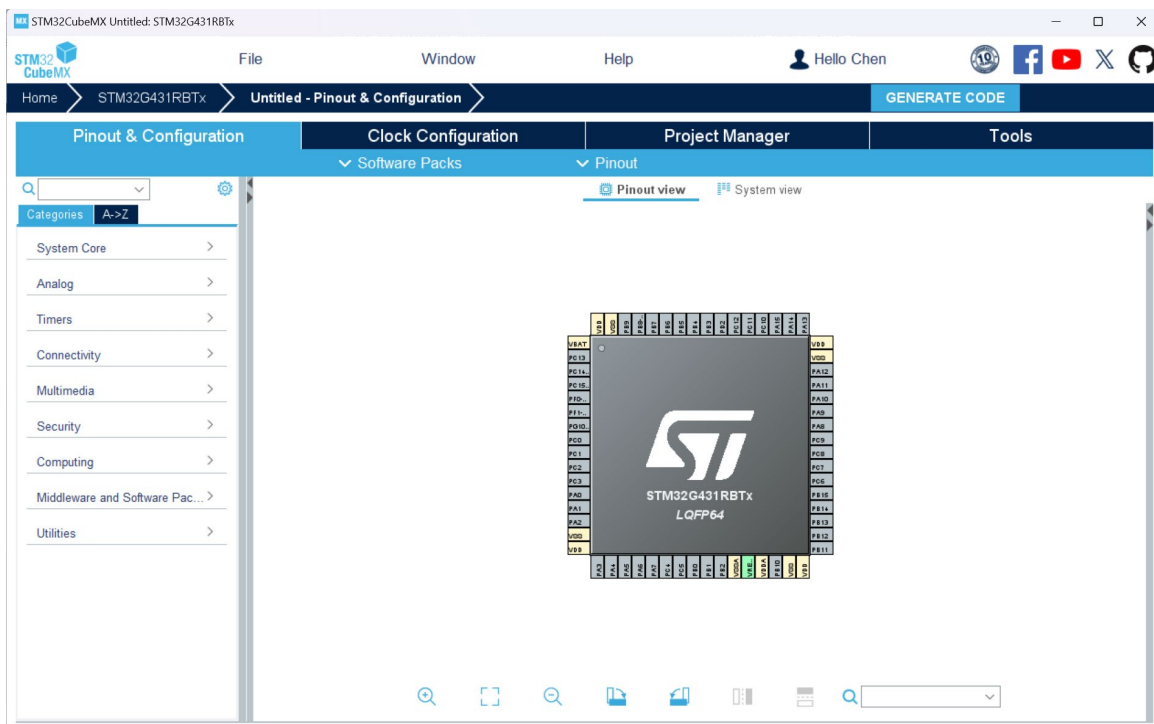


图2.4 STM32CubeMX工程界面

(5) 对工程进行简单的配置，确保生成的工程程序可下载至单片机。

① 选择System Core -> SYS，修改芯片下载方式（Debug）式为SWD（Serial Wire），此步骤必须选择，否则会影响后续的程序下载。

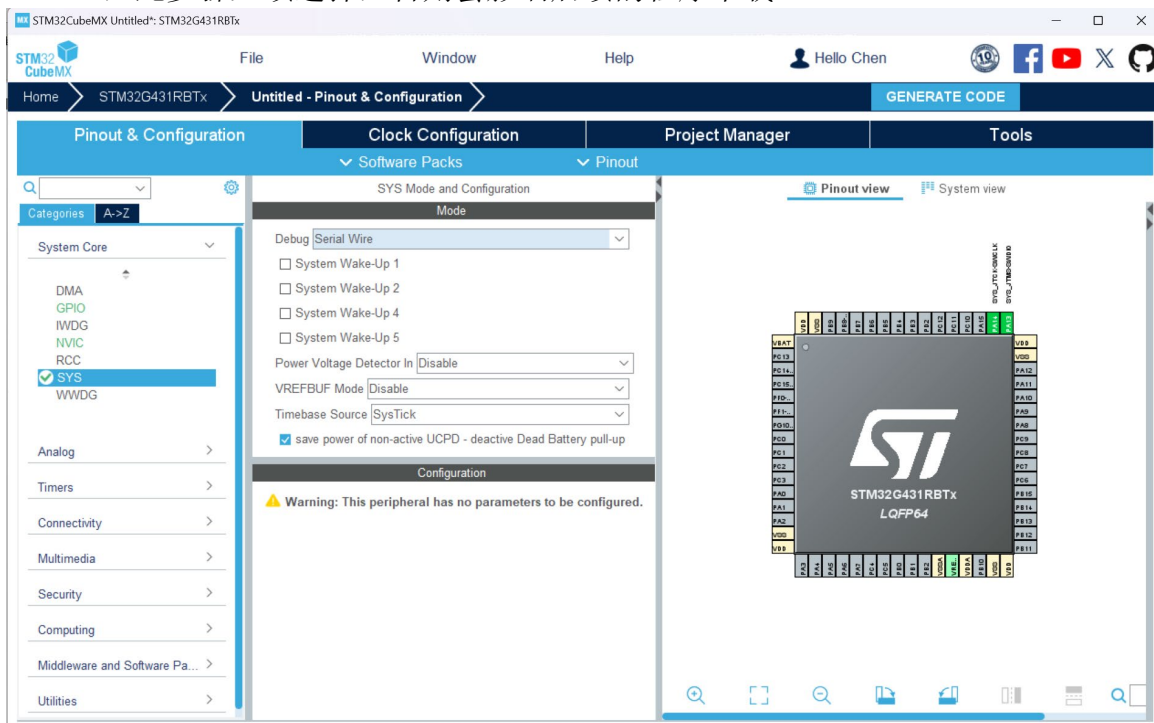


图2.5 修改下载方式

② 修改时钟源，选择 System Core -> RCC，修改高速晶振（HSE）方式为片外晶振（Crystal/Ceramic Resonator）。

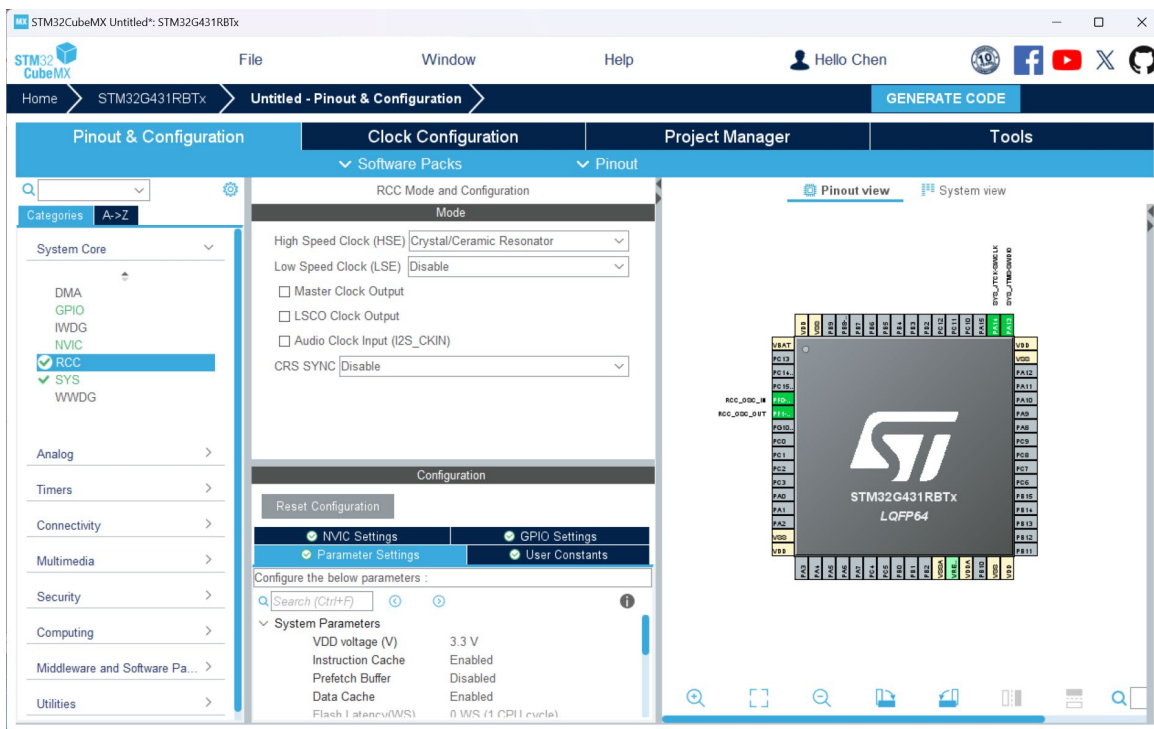


图2.6修改时钟源

③ 修改时钟树，点击界面上的 Clock Configuration选项卡，根据实验一绘制的原理图可以知道，开发板所用外部晶振频率为24MHz，修改时钟树的Input frequency为24；修改PLL Source Mux到HSE，启用外部晶振；在主频（HCLK）修改系统时钟频率到170，输入完毕并按回车后，软件提示要计算不合适的参数，点击OK，软件会自动计算各寄存器的分频/倍频系数，若没有自动计算出合适的参数，参考下图中的参数进行时钟树配置。

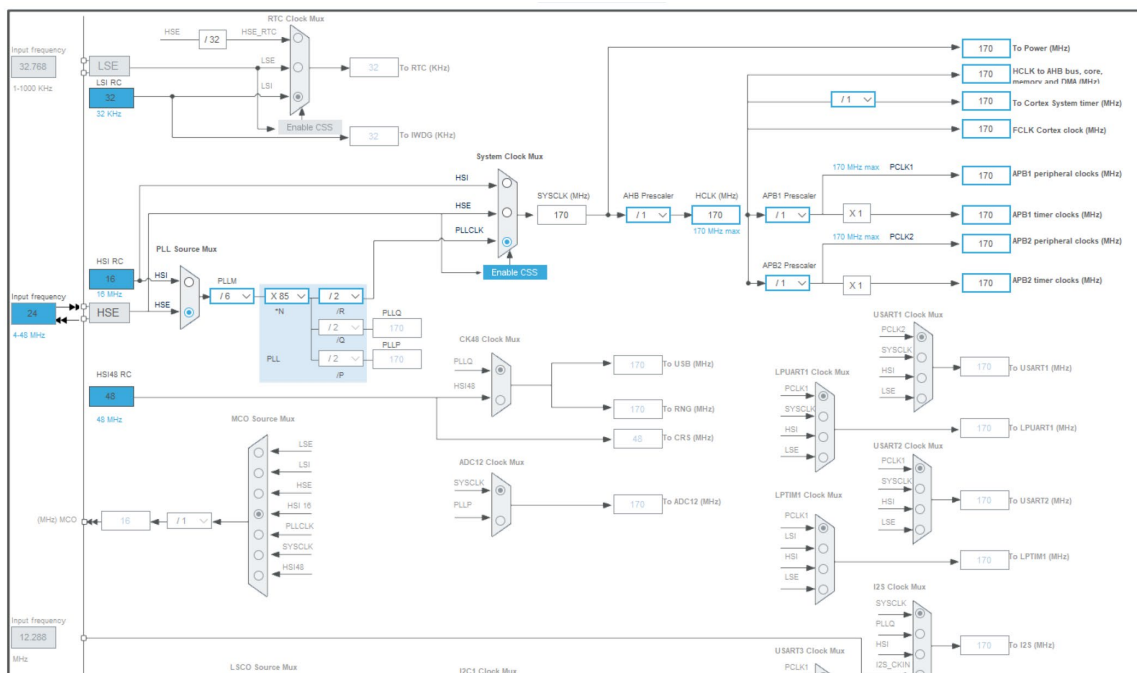


图2.7 修改时钟树

④ 配置IO口用来查看程序运行效果。从开发板的原理图中可以看出，LED灯的控制经过了一个74HC573锁存器，LE（PD2）为高电平时，Q端跟随D端变化，所以在这里初始化2个IO口，分别是LD1（PC8）和LE（PD2）。

回到Pinout & Configuration选项卡，在右侧的Pinout view中分别找到PC8和PD2，点击对应引脚，选择GPIO_Output模式。

选择 System Core -> GPIO，在Configuration中可以看到多了一个GPIO配置的选项卡，选中PD2，修改PD2的GPIO output level为high；PC8引脚的参数不用更改。

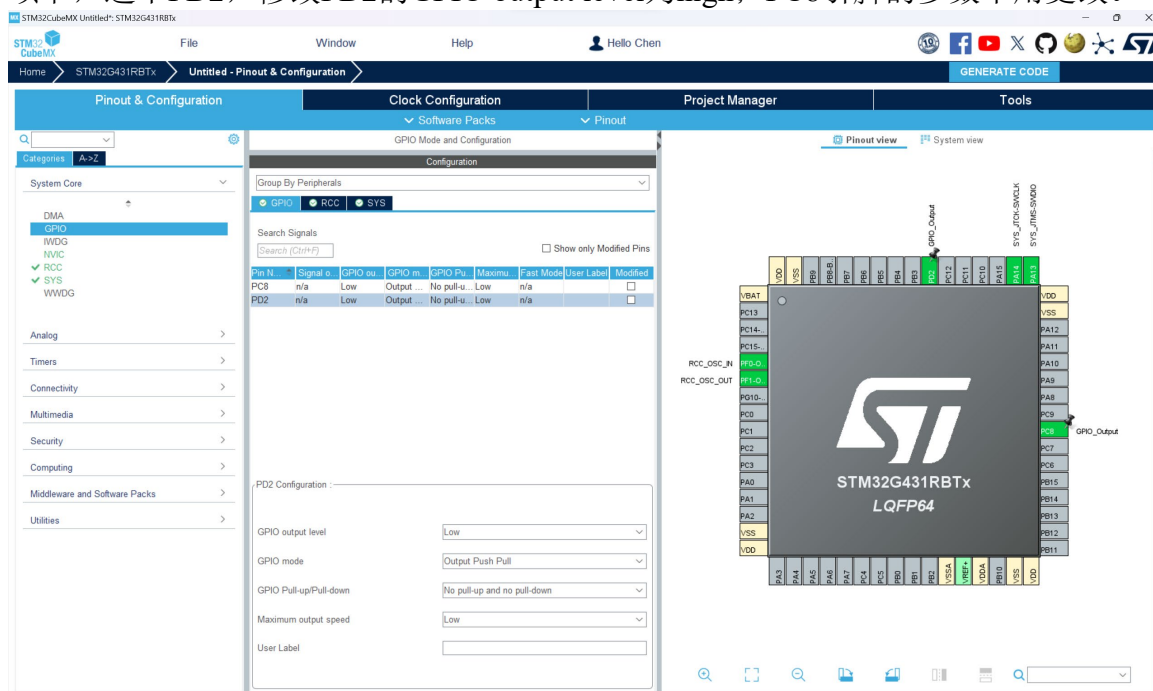


图2.8 配置GPIO口

（6）修改工程配置

① 点击Project Manager选项卡，在Project选项卡中分别输入工程名（例如:exp1），保存路径默认即可；修改所用的IDE（Toolchain/IDE）为MDK-ARM，版本为：V5.32（默认）。

在Firmware Package Name and Version 中，取消勾选Use Latest available version，在下拉菜单中选择库版本为1.6.0（STM32Cube FW_G4_V1.6.0）。

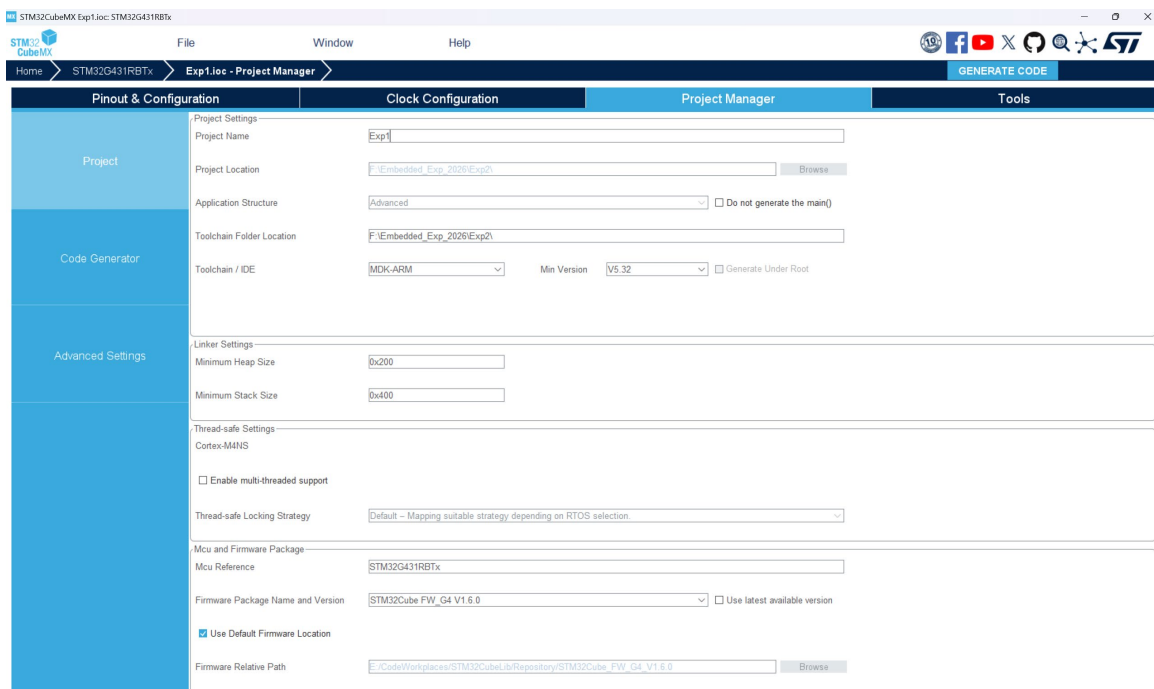


图2.9 配置工程参数

② 点击 Code Generator选项卡，选择copy only the necessary library files，勾选 Generate peripheral initialization as a pair of ‘.c/.h’ files per peripheral。

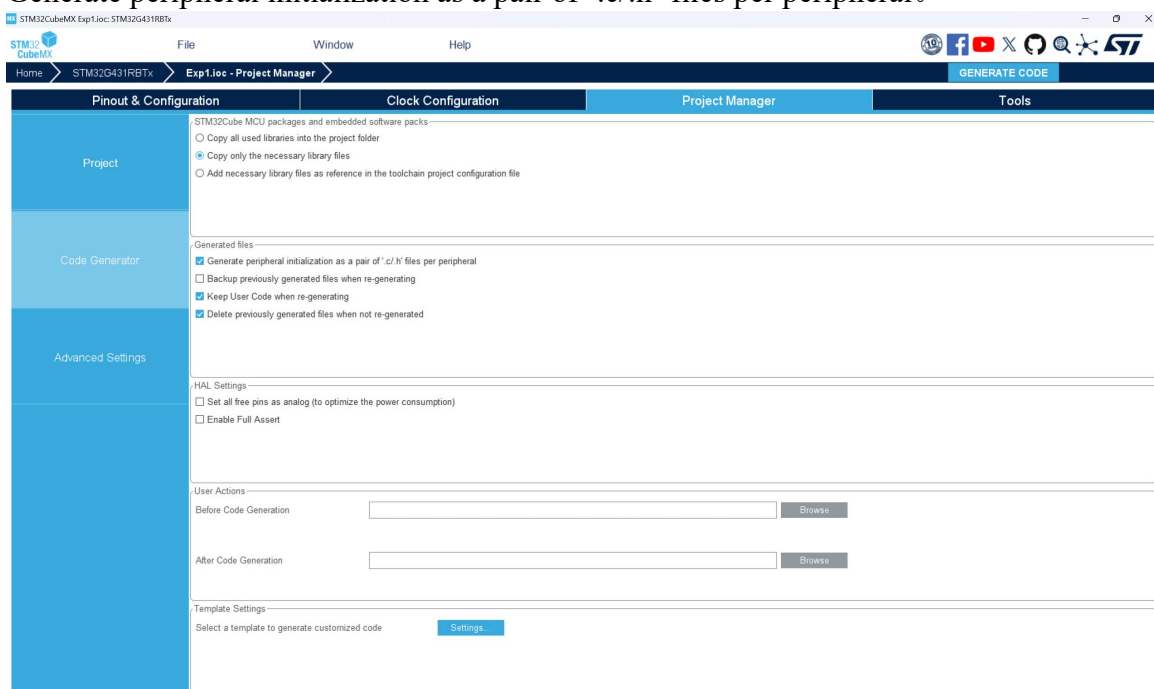


图2.10 配置代码生成参数

③ 点击GENERATE CODE按钮，生成工程。

(7) 代码生成后，点击Open Floder按钮打开工程所在文件夹，熟悉文件夹结构，进入MDK-ARM文件夹，双击exp.uvprojx，通过Keil软件打开工程，熟悉软件

的各个区域功能（菜单栏，工程栏，编辑区，信息输出区），软件界面如图所示：

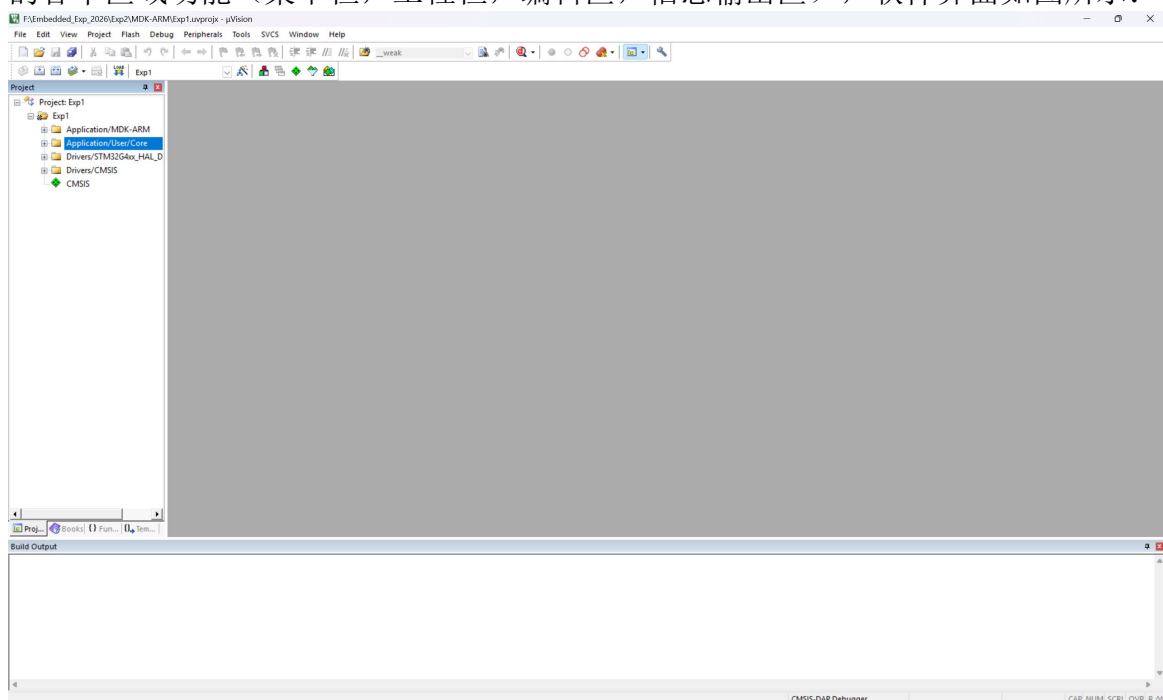


图2.11 Keil软件界面

（8）修改Keil工程下载参数。在工作空间（workplace）中选中工程名，点击 Options for Target...，弹出工程配置参数的界面

① 在界面中点击Debug选项卡，修改调试器为 CMSIS-DAP Debugger

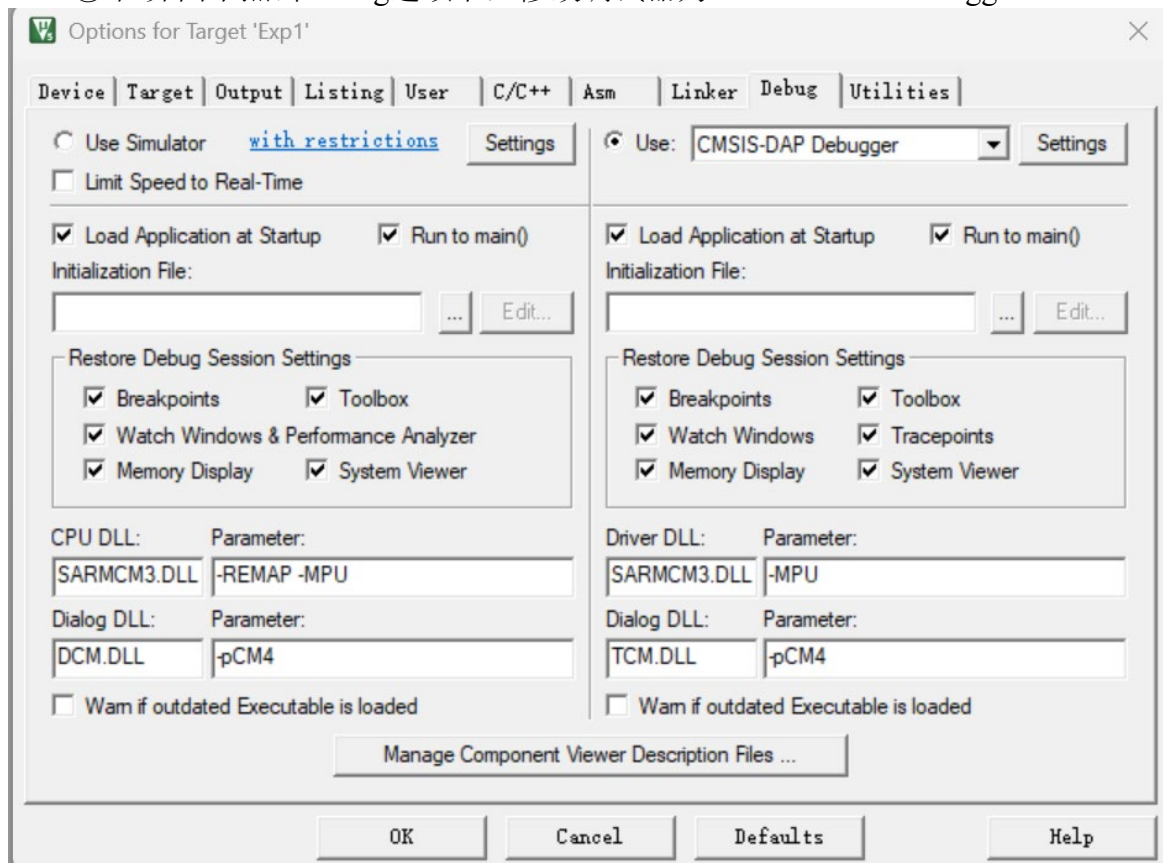


图2.12 下载配置界面

② 点击CMSIS-DAP右侧的Settings，确保调试器链接模式为SW，如下图所示。

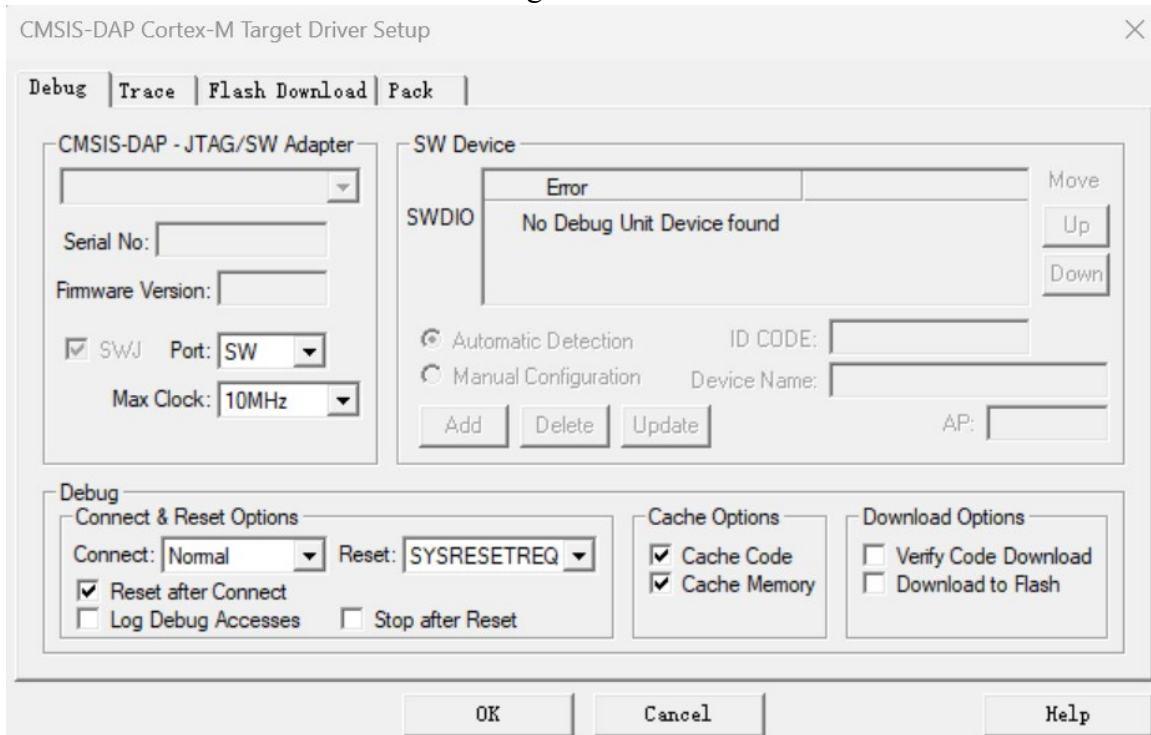
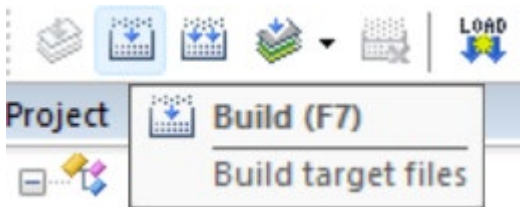


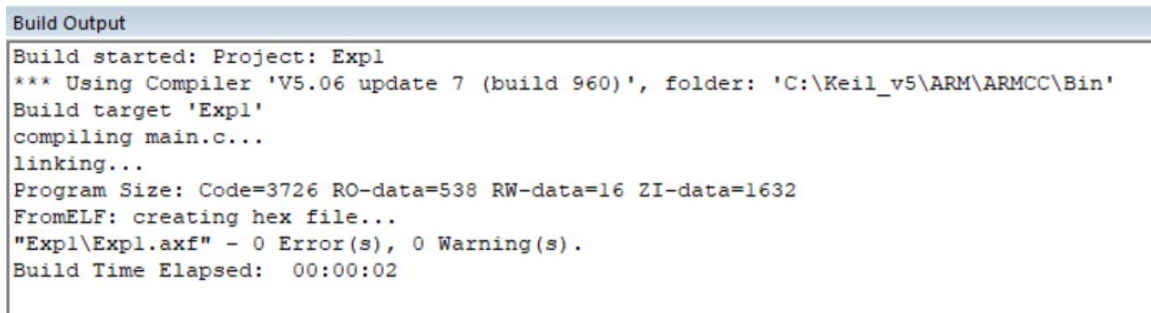
图2.13 下载器配置界面

③ 修改完成后点击OK按钮保存更改。

(9) 编译工程，检查工程创建是否有错误。点击菜单栏的build按钮

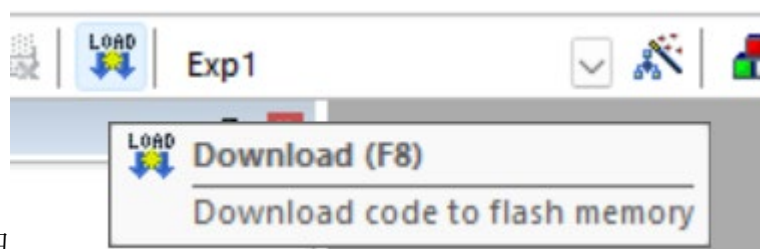


，并查看输出信息是否提示有错误，如图所示



即为工程创建完成。

(10) 下载程序到开发板。通过USB线将开发板连接到电脑，点击菜单栏的



Download按钮，若无任何报错，则下载成功，进入调试界面。点击新界面上的run按钮，开发板的第一个LED灯亮起。

2.3.2 编写一个LED流水灯程序

(1) 开发板LED灯电路的原理图如下：

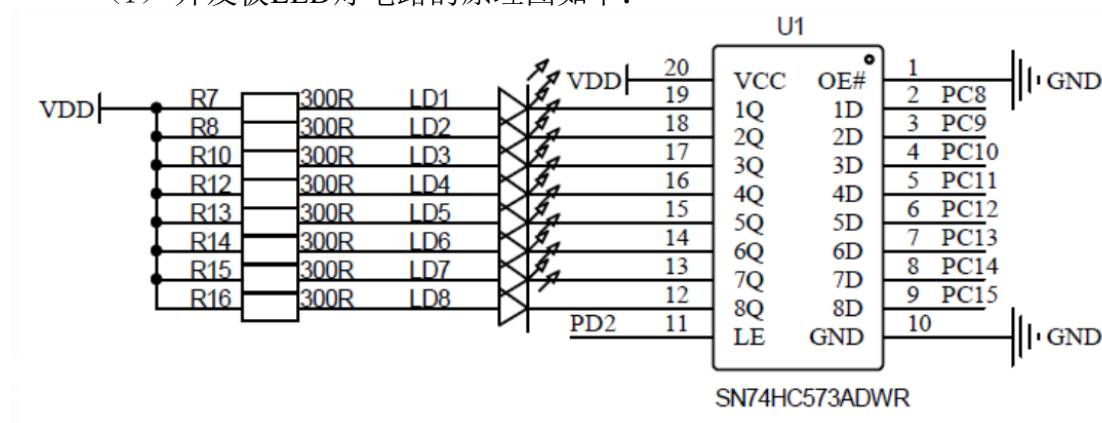


图2.14 LED灯电路原理图

- (2) 在上一步创建的STM32CubeMX工程基础上，初始化PC8~PC15，PD2引脚为GPIO输出引脚，初始化PB0，PB1，PB2，PA0为输入引脚。
- (3) 生成Keil工程并打开，查看stm32g4xx_hal_gpio.c/h文件，熟悉hal库提供的驱动函数。
- (4) 在main.c文件中，找到main函数，利用hal库函数提供的功能代码，编写LED流水灯的代码实现如下功能：8个LED灯依次点亮，再依次熄灭，每个灯切换间隔为500ms，下载到开发板并查看代码运行效果。
- (5) 将修改后的main函数及自己编写的函数内容写在实验报告中，并标注不少于50%的注释内容。
- (6) 在实验报告中分析STM32单片机在本次实验中使用GPIO端口时使用到的寄存器，并分析相关寄存器的具体功能。

2.4 实验思考题

- 1、请分析生成的STM32工程目录。
- 2、请对比STM32单片机的IO口与51单片机的IO口在结构、功能上的差异。

实验3 UART异步串行通信

3.1 实验目的

了解 STM32 Cortex-M4系列芯片的软件开发流程
掌握 STM32 UART的使用方法

3.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

3.3 实验流程

(1) 开发板UART->USB电路的原理图如下:

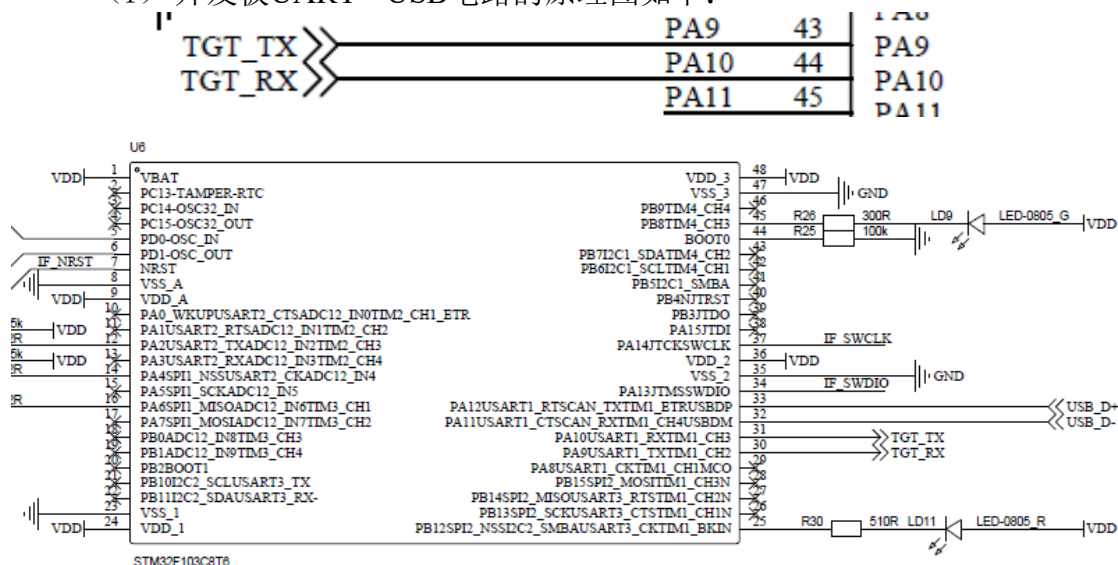


图4.1 UART->USB电路原理图

- (2) 复制一份实验二的CubeMX工程到新的位置，在实验二的STM32CubeMX工程基础上，初始化PA9和PA10引脚对应的模块，115200波特率，8位数据位，1位停止位，无校验位。
- (3) 生成Keil工程并打开，查看生成的USART驱动相关代码。
- (4) 在main.c文件中，利用hal库函数提供的功能代码，编写如下功能：
 - 1) 在串口中断函数的中实现串口数据的接收功能，并将接收到的数据发送出去。
 - 2) 通过串口助手从电脑发送字符，单片机根据接收到的数据，实现以下功能
 - ①若接收到字符串“M1”，则流水灯按实验二的方式运行；
 - ②若接收到字符串“M2”，所有灯默认熄灭，若通过串口接收到字符1-8，切换对应L1-L8灯的状态
 - 3) *通过串口输出单片机运行的时间(s)。
- (5) 将修改后的main函数、中断函数以及自己编写的函数内容写在实验报告中，并标注不少于50%的注释内容。
- (6) 在实验报告中分析STM32单片机在本次实验中使用USART模块时使用

到的寄存器，并分析相关寄存器的具体功能。

3.4 实验思考题

1. 典型的串口通信有哪几种标准，并说出这几种标准的不同点和应用场景（例：RS232）。

实验4 GPIO应用开发1

4.1 实验目的

了解 STM32 Cortex-M4系列芯片的软件开发流程
掌握 STM32 GPIO控制LCD屏幕的方法

4.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

4.3 实验流程

- (1) 本次实验利用GPIO产生可以控制LCD屏幕的时序，将指定内容显示到LCD屏幕上，LCD接口电路原理图如下图所示：

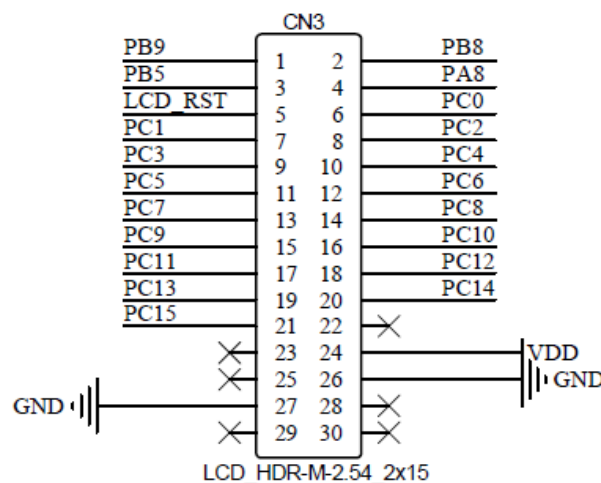


图5.1 LCD屏幕接口电路原理图

- (2) 复制一份实验三的CubeMX工程到新的位置，在实验三的STM32CubeMX工程基础上，添加LCD控制引脚相关GPIO的初始化，将所有引脚初始化为GPIO Output的模式；
- (3) 生成Keil工程并打开，添加本次实验提供的三个驱动文件到工程中（lcd.c / lcd.h / fonts.h）。
- (4) 在main.c文件中，找到main函数，利用hal库函数以及LCD驱动文件提供的功能代码，编写如下功能：
- 1) 根据提供的驱动代码，初始化LCD屏幕；
 - 2) 在屏幕任意位置显示自己姓名的拼音以及学号；
 - 3) 在屏幕任意位置绘制一条直线（固定显示），一个矩形（每100ms水平移动1次）、和一个圆形（每100ms垂直移动一次）。
 - 4) *通过串口助手发送长度不大于32个字符的字符串（只包含英文、数字和英文标点），单片机将接收到的字符串显示到LCD屏幕

的固定位置上。

- (5) 将修改后的main函数、中断函数以及自己编写的函数内容写在实验报告中，并标注不少于50%的注释内容。
- (6) 根据提供的数据手册和LCD屏幕驱动代码绘制出LCD读/写数据/命令的时序图

4.4 实验思考题

1. 根据提供的LCD屏幕手册，若要显示一幅彩色图片到屏幕上，需要占用单片机多大的存储空间，如果将屏幕分辨率提升至800*480，在STM32G431RBT6芯片中能否存储1副图片的内容，请列出计算过程。

实验5 GPIO应用开发2

5.1 实验目的

了解 STM32 Cortex-M4系列芯片的软件开发流程
掌握 STM32 的GPIO产生I2C控制时序的方法

5.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

5.3 实验流程

- (1) 掌握SystemTick模块的使用（HAL_Delay）。
- (2) 本次实验利用GPIO产生I2C通讯时序，控制存储芯片24C02存储数据，相关电路如图所示。

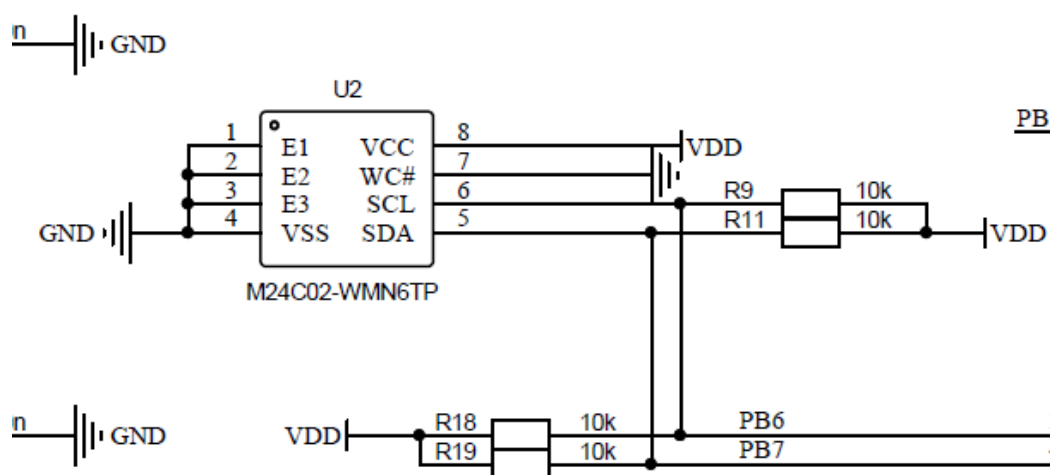


图6.1 LCD屏幕接口电路原理图

- (3) 复制一份实验四的CubeMX工程到新的位置，在实验四的STM32CubeMX工程基础上，添加I2C控制引脚相关GPIO的初始化，将所有引脚初始化为GPIO Output的模式；
- (4) 生成Keil工程并打开，添加本次实验提供的两个驱动文件到工程中（i2c_hal.c / i2c_hal.h）。
- (5) 在main.c文件中，利用hal库函数以及I2C驱动文件提供的功能代码，编写如下功能：
 - 1) 参考24C02数据手册，利用I2C驱动代码实现数据的存储和读写；
 - 2) 在地址0存储单片机的开机次数（第一次编写程序需将此处清零），将开机次数显示到LCD屏幕上；
 - 3) 在地址1、地址2存储学号的后四位，将数据显示到LCD屏幕上；

- 4) *在地址3存储串口发送到单片机的ASCII码值（1位），并将该数据显示到LCD屏幕上。
- (6) 将修改后的main函数、中断函数以及自己编写的函数内容写在实验报告中，并标注不少于50%的注释内容。
- (7) 在实验报告中绘制出I2C通讯协议读/写数据/命令的时序图，并分析24C02存储模块的读写流程。

5.4 实验思考题

1. 请比较本次实验使用的EEPROM存储芯片和单片机内部存储的FLASH在存储数据过程中的优缺点各是什么。
2. 如果使用STM32内部的FLASH存储器存储数据，能否通过编程实现？如果可以，在使用过程中需要注意哪些问题。

实验6 可编程延迟模块（基本定时功能）

6.1 实验目的

了解 STM32 Cortex-M4系列芯片的软件开发流程
掌握 STM32 定时器TIM的基本使用方法

6.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

6.3 实验流程

- (1) TIM模块是STM32芯片的一个内部模块，无对应电路图。
- (2) 复制一份实验五的CubeMX工程到新的位置，在实验五的STM32CubeMX工程基础上，添加任意两个TIM模块的初始化，初始化为基本定制器模式。
- (3) 生成Keil工程并打开，查看生成的TIM模块驱动相关代码。
- (4) 在main.c文件中，利用hal库函数以及LCD驱动文件提供的功能代码，编写如下功能：
 - 1) 利用一个定时器产生10ms中断，在LCD屏幕上显示一个格式为HH: MM: SS的时间，该时间可以通过串口发送格式为(HMS)的十六进制字符串修改时间（长度为3），其中H: 0~23, M: 0~59, S:0~59;
 - 2) *利用另一个定时器产生1ms中断，在LCD屏幕上显示一个格式为MM: SS: MS的秒表，其中B1按键对应秒表启动，B2按键对应秒表暂停，B3按键对应秒表暂停并清零;
- (5) 将修改后的main函数、中断函数以及自己编写的函数内容写在实验报告中，并标注不少于50%的注释内容;
- (6) 在实验报告中分析STM32单片机使用TIM定时器模块作为基本定时器时配置的相关寄存器功能，分析定时时间和时钟树的关系;

6.4 实验思考题

1. 请简述STM32中通用定时器的工作原理及可以实现的功能。
2. 请从定时器的原理、定时时间长度、定时器功能几个方面分析51单片机定时器与STM32定时器。

实验7 可编程延迟模块（频率输出及输入捕获）

7.1 实验目的

了解 STM32 Cortex-M4系列芯片的软件开发流程
掌握 STM32 定时器TIM的高级使用方法

7.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

7.3 实验流程

- (1) TIM模块是STM32芯片的一个内部模块，无对应电路图。
- (2) 复制一份实验六的CubeMX工程到新的位置。
- (3) 生成Keil工程并打开。
- (4) 在main.c文件中，利用hal库函数以及LCD驱动文件提供的功能代码，编写如下功能：
 - 1) TIM1和TIM3分别通过CH1N（PA7）和CH1（PA6）输出2KHz和1KHz的矩形波；
 - 2) TIM2通过CH2（PA1）输入TIM1或者TIM3产生的矩形波，用CH1和CH2测量矩形波的周期和脉冲宽度；
 - 3) TIM1或TIM3产生矩形波的脉冲宽度可以通过按键B2改变；
- (5) 验证代码功能：
 - 1) 用导线连接J3-4（PA1）和J3-9（PA6），查看频率和脉宽；
 - 2) 用导线连接J3-4（PA1）和J3-10（PA7），查看频率和脉宽，按下按键B2修改输出的周期和脉宽，查看显示内容；
 - 3) 用导线连接J3-4（PA1）和J9-2（或J10-2），旋转R39（或R40），FRE的值大约在800~22000（Hz）之间变化。
- (6) 将修改后的main函数、中断函数以及自己编写的函数内容写在实验报告中，并标注不少于50%的注释内容；
- (7) 在实验报告中分析STM32单片机使用TIM定时器模块作为PWM波形输出以及输入捕获功能时比基本的定时模式额外使用了哪些寄存器，分析相关寄存器的功能。

7.4 实验思考题

1. 请简述PWM波的应用场合有哪些。
2. 请查阅资料并分析常用的航模遥控器信号有哪些类型，如果使用STM32芯片用于航模遥控器信号的接收端，需要使用单片机的哪些功能模块。

实验8 数模转换模块

8.1 实验目的

了解 STM32 Cortex-M4系列芯片的软件开发流程
掌握 STM32 A/D转换模块的使用方法

8.2 实验设备及器件

计算机
Keil for ARM
STM32CubeMX

8.3 实验流程

- (1) 实验板上有两个电位器连接到了STM32的ADC引脚上，相关电路原理图如下。

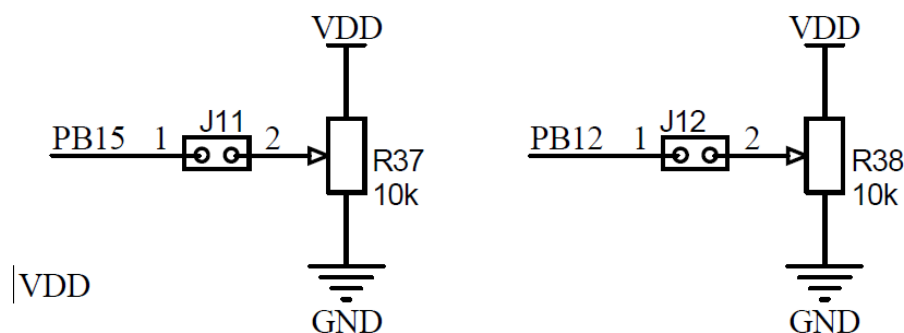


图8.1 电位器接口电路原理图

- (2) 复制一份实验七的CubeMX工程到新的位置，在实验七的STM32CubeMX工程基础上，初始化PB12和PB15两个引脚对应的ADC通道。
- (3) 生成Keil工程并打开，查看生成的ADC模块驱动相关代码。
- (4) 在main.c文件中，利用hal库函数以及LCD驱动文件、EEPROM驱动文件提供的功能代码，编写如下功能：
- 1) 在LCD屏幕上分别显示PB15和PB12读取的电位器电压值。
 - 2) 参考实验3（M1和M2功能还需存在），增加指令“M3”：当电脑通过串口助手发送字符‘1’，单片机通过串口发送PB15引脚的电压值（例：2.45V），发送字符‘2’，返回PB12引脚的电压值，发送字符‘3’，返回两个引脚的电压值。
 - 3) 在M3上继续增加功能，若收到字符“1”，则8个LED按PB15的电压值的大小点亮（0V对应全灭，3.3V对应全亮），若收到字符“2”，LED用于显示PB12的电压值，若收到字符“3”，L1-L4显示PB15电压（0V对应全灭，3.3V对应全亮），L5-L8显示PB12电压。
- (5) 将修改后的main函数内容写在实验报告中，并标注不少于50%的注释内容。
- (6) 在实验报告中分析STM32单片机使用ADC模块时配置的相关寄存器功能。

8.4 实验思考题

1. 请简述模数转换模块的主要参数。
2. 请比较ADC的单端输入和差分输入的优缺点。